

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP.HCM
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO TRANG WEB CÁC ỨNG DỤNG CỦA
STACK

Mã môn học: DASA230179

Giáo viên HD: Th.S Vũ Đình Bảo

Năm học 2025-2026

Mục lục

1	GIỚI THIỆU	3
2	NỘI DUNG	4
2.1	Giới thiệu về Stack	
2.2	Đệ quy (Recursion)	
2.3	Chuyển từ biểu thức Infix sang Postfix	
2.4	Kiểm tra dấu ngoặc	
2.5	Đổi cơ số	
3	KẾT LUẬN	14
3.1	Các kết quả đạt được	
3.2	Hướng phát triển trong tương lai	
	PHỤ LỤC	15
	Bảng viết tắt	
	Danh sách hình ảnh	
	Mã nguồn chương trình	
	Hình ảnh minh họa	

GIỚI THIỆU

Trang web được xây dựng bằng **Python + Streamlit** để minh họa các ứng dụng của ngăn xếp (stack). Bài báo cáo này tập trung vào việc giải thích ý tưởng và đưa ảnh minh họa kết quả chạy demo, sau cùng là mã nguồn được để trong phụ lục kèm theo cấu trúc thư mục của nó.

NỘI DUNG

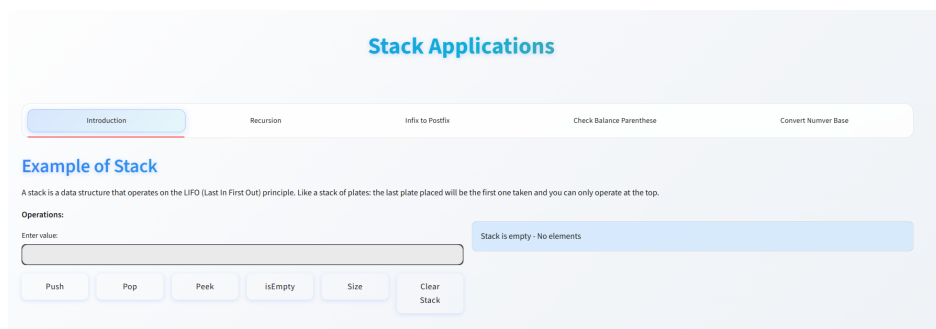
2.1 Giới thiệu về Stack

Trong ví dụ này, ứng dụng minh họa cách hoạt động của **Stack** (ngăn xếp). Stack là một cấu trúc dữ liệu tuân theo nguyên tắc *LIFO* (Last In, First Out), tương tự như chồng đĩa: đĩa nào đặt vào sau cùng sẽ được lấy ra trước tiên, và ta chỉ có thể thao tác ở đỉnh của ngăn xếp.

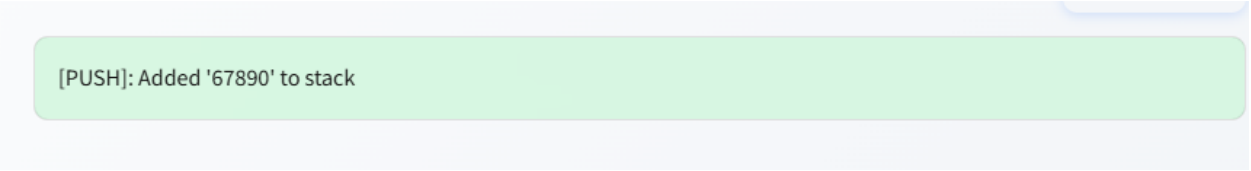
Các thao tác chính

Ứng dụng cho phép người dùng nhập dữ liệu và thực hiện các phép toán cơ bản trên Stack, được minh họa ở hình 2.1 bao gồm:

- **Push:** Thêm phần tử mới vào đỉnh stack.
- **Pop:** Lấy phần tử ở đỉnh stack ra ngoài.
- **Peek:** Xem giá trị phần tử đang ở đỉnh mà không xóa nó.
- **isEmpty:** Kiểm tra stack có rỗng hay không.
- **Size:** Trả về số lượng phần tử hiện có trong stack.
- **Clear Stack:** Xóa toàn bộ phần tử trong stack.



Hình 2.1: Hình minh họa trang Web



```
[PUSH]: Added '67890' to stack
```

Hình 2.2: Thông báo log khi Push vào Stack



Hình 2.3: Thông tin về Stack

Ví dụ thao tác Push

Khi nhập giá trị và nhấn nút **Push**, hệ thống sẽ thực hiện logging giống hình 2.2:

- Ghi lại thông báo trong phần logging (ví dụ: **[PUSH]: Added '67890' to stack**).
- Cập nhật trạng thái stack, hiển thị chi tiết phần tử mới.

Chi tiết phần tử trong Stack

Mỗi phần tử trong stack được hiển thị với các thông tin, minh họa ở hình 2.3:

- **Value:** Giá trị được lưu.
- **Index:** Vị trí của phần tử trong stack.
- **Position:** Cho biết phần tử có phải Top của stack không.
- **Size:** Kích thước bộ nhớ của phần tử (tính bằng bytes).
- **Address:** Địa chỉ trong bộ nhớ.

> Element 5 (TOP)

> Element 4

> Element 3

> Element 2

> Element 1

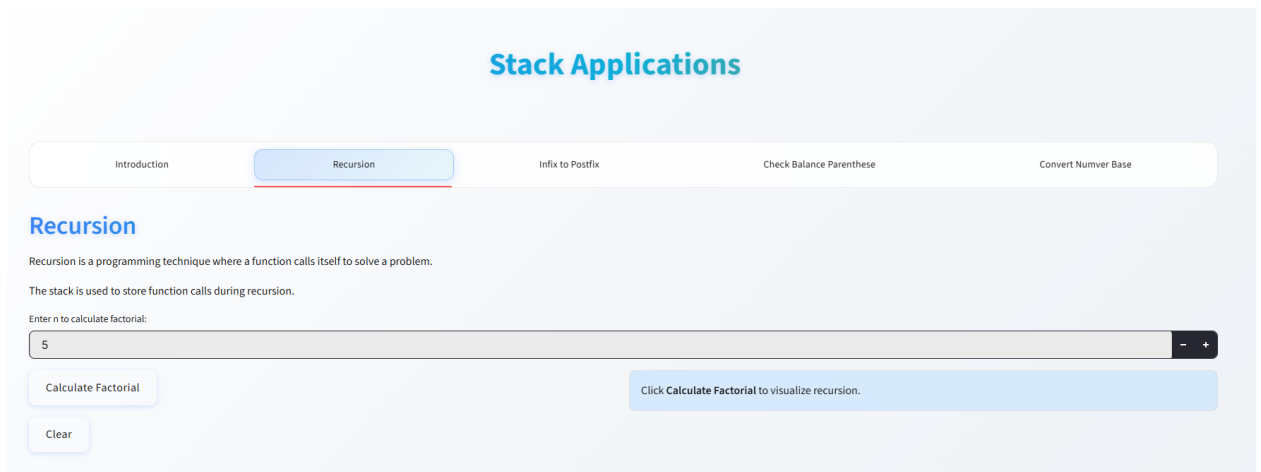
Hình 2.4: Xếp chồng các phần tử trong Call Stack

Khi người dùng thực hiện liên tiếp nhiều thao tác **Push**, các phần tử sẽ được xếp chồng lên nhau theo nguyên tắc **LIFO** (Last In – First Out). Phần tử mới nhất luôn nằm ở vị trí *TOP*, trong khi các phần tử cũ hơn được đẩy xuống dưới. Điều này được minh họa ở Hình 2.4, nơi ta thấy các phần tử được sắp xếp tuần tự như một chồng đĩa. Ngược lại, khi thực hiện thao tác **Pop**, phần tử ở đỉnh (*TOP*) sẽ được lấy ra trước tiên. Sau khi phần tử này bị loại bỏ, phần tử ngay phía dưới sẽ trở thành *TOP* mới. Quá trình này tiếp tục cho đến khi ngăn xếp rỗng.

2.2 Đệ quy (Recursion)

Trong demo thứ hai, hệ thống minh họa nguyên lý hoạt động của **đệ quy** (recursion). Đệ quy là kỹ thuật lập trình trong đó một hàm tự gọi lại chính nó để giải quyết một bài toán. Khi sử dụng đệ quy, **ngăn xếp (stack)** được dùng để lưu trữ các lời gọi hàm (function calls) cho đến khi quay ngược trở lại.

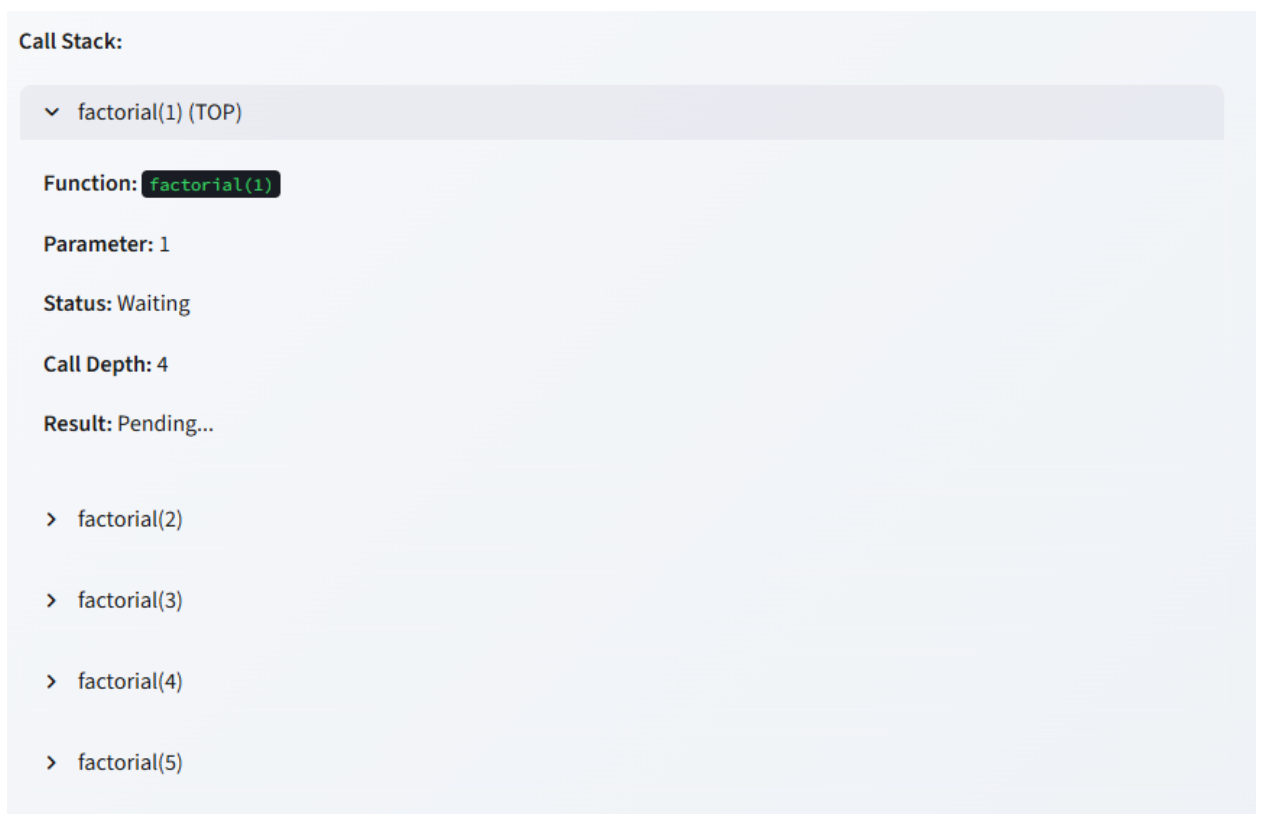
Ở đây, chương trình minh họa bằng ví dụ tính **giai thừa (factorial)** của một số nguyên n . Người dùng nhập giá trị n vào ô nhập liệu, sau đó nhấn nút *Calculate Factorial* để bắt đầu. Mỗi lần hàm gọi đệ quy sẽ được đưa vào stack, và khi hoàn thành sẽ lần lượt được lấy ra theo cơ chế LIFO. Điều này được minh họa ở Hình 2.5, trong đó người dùng nhập $n = 5$



Hình 2.5: Giao diện đệ quy

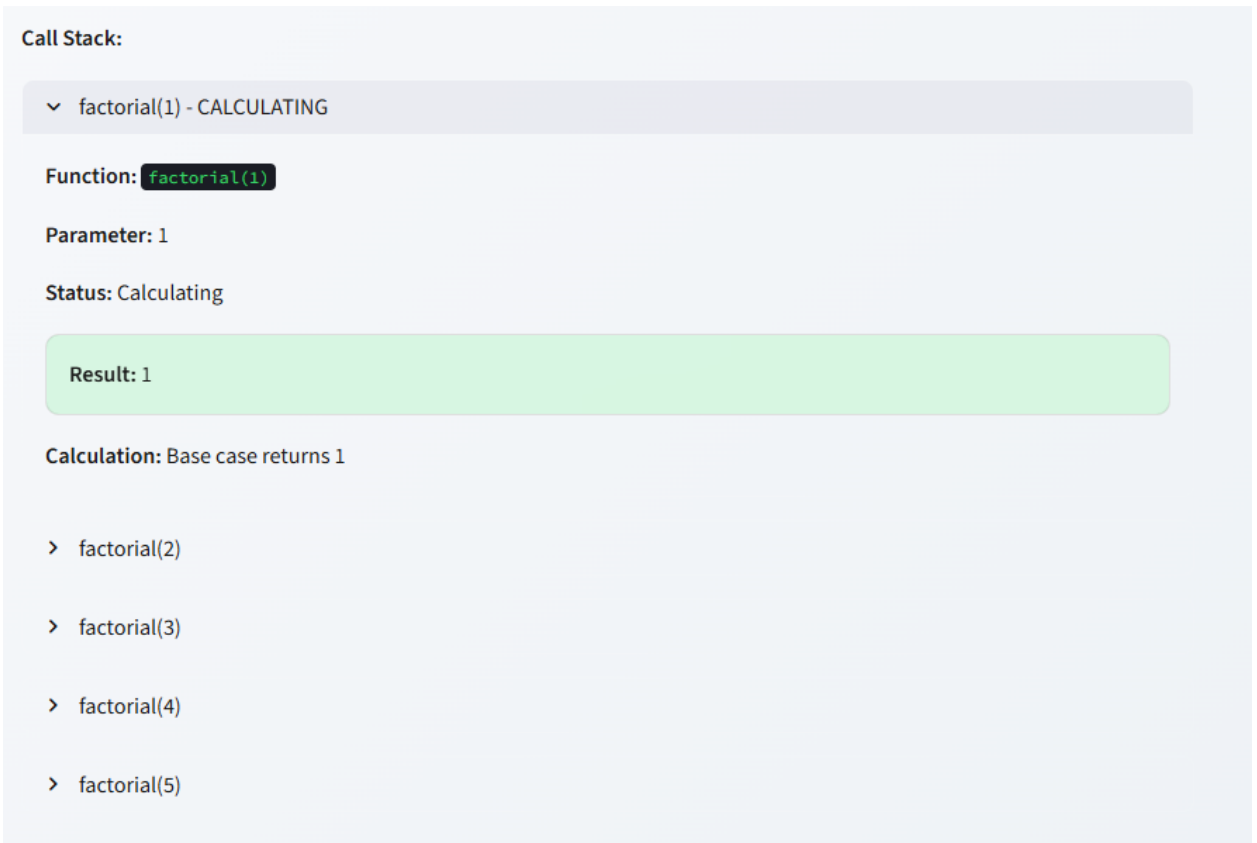
để tính $5!$ và hệ thống chuẩn bị biểu diễn quá trình gọi hàm đệ quy.

Sau khi nhấn *Calculate Factorial*, chương trình tiến hành **xây dựng call stack**. Mỗi lời gọi hàm `factorial(n)` sẽ được đưa vào stack, từ `factorial(5)` cho đến `factorial(1)`. Điều này được minh họa trong Hình 2.6, với `factorial(1)` hiện đang nằm ở đỉnh (TOP) của stack.



Hình 2.6: Giai đoạn 1: Xây dựng call stack cho bài toán tính $5!$

Trong giai đoạn này, các hàm được đưa vào stack nhưng chưa được thực thi đầy đủ (*Status: Waiting*, *Result: Pending*). Chỉ khi gặp điều kiện dừng ở `factorial(1)` thì tiến trình **unwinding** (tháo ngược stack) mới bắt đầu.



Hình 2.7: Độ quy đạt được điều kiện cơ sở: $factorial(1) = 1$

Khi đạt tới **điều kiện cơ sở** $factorial(1)$, hàm bắt đầu được tính toán thay vì chờ đợi. Kết quả $factorial(1) = 1$ được trả về, như minh họa ở Hình 2.7. Tại thời điểm này, tiến trình **unwinding** bắt đầu:

- Kết quả của $factorial(1)$ được trả về cho $factorial(2)$.
- Sau đó, $factorial(2)$ sẽ tính $2 \times factorial(1)$ và trả kết quả lên $factorial(3)$.
- Quá trình tiếp tục cho đến khi tính xong $factorial(5)$.

2.3 Chuyển từ biểu thức Infix sang Postfix

Trong toán học, biểu thức thường được viết ở dạng **Infix** (ví dụ: $3 + 4 \times 2$), nhưng máy tính lại dễ dàng xử lý hơn ở dạng **Postfix**. Cấu trúc dữ liệu **Stack** trong trường hợp này được sử dụng để:

- Duyệt qua biểu thức Infix và xác định thứ tự ưu tiên của các toán tử.
- Chuyển đổi biểu thức sang Postfix bằng cách lưu tạm toán tử trên Stack.
- Tính toán giá trị biểu thức Postfix theo từng bước bằng cách push và pop các toán

hạng.

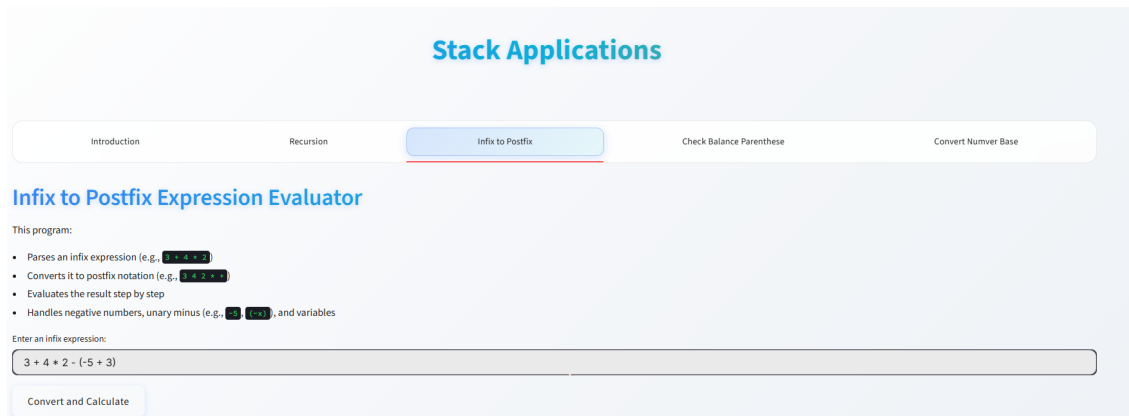
Ví dụ: biểu thức Infix

$$3 + 4 \times 2 - (-5 + 3)$$

sẽ được chuyển thành Postfix:

$$3\ 4\ 2\ \times\ +\ -\ 5\ 3\ +\ -$$

Giao diện chính được thể hiện trong Hình 2.8.



Hình 2.8: Chuyển đổi từ Infix sang Postfix bằng Stack

Bước 1: Tokenization

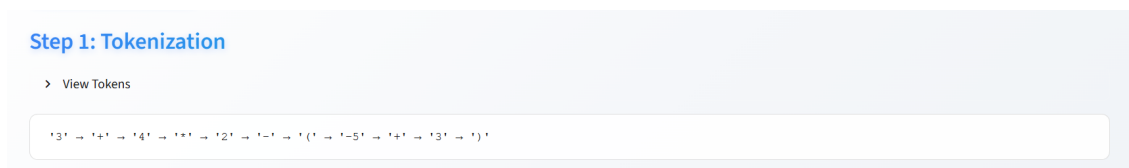
Trước khi thực hiện chuyển đổi, biểu thức Infix cần được phân tích thành các **token**. Token là các đơn vị nhỏ như số, toán tử, hoặc dấu ngoặc.

Ví dụ, với biểu thức:

$$3 + 4 \times 2 - (-5 + 3)$$

Hệ thống sẽ tách thành chuỗi các token:

'3' → '+' → '4' → '*' → '2' → '-' → '(' → '-5' → '+' → '3' → ')'



Hình 2.9: Bước 1: Phân tích biểu thức thành các token

Bước 2: Chuyển đổi Infix $\rightarrow$ Postfix

Sau khi có các token, ta sử dụng thuật toán **Shunting-yard** để chuyển đổi. Ý tưởng chính:

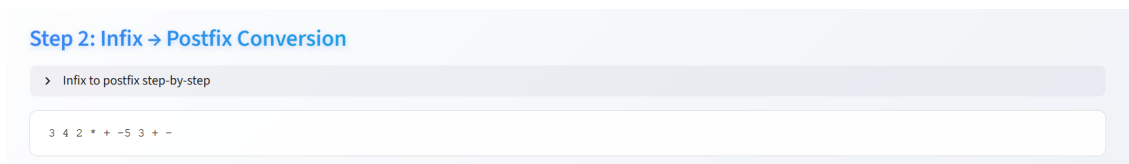
- Nếu là toán hạng (số, biến) $\rightarrow$ đưa trực tiếp vào kết quả.
- Nếu là toán tử (+, -, *, /) $\rightarrow$ đưa vào *stack*, tuân theo độ ưu tiên và tính kết hợp.
- Nếu gặp dấu ngoặc '(' $\rightarrow$ đưa vào stack.
- Nếu gặp dấu ngoặc ')' $\rightarrow$ pop toán tử từ stack ra kết quả cho đến khi gặp '('.
- Sau khi duyệt hết biểu thức $\rightarrow$ pop toàn bộ toán tử còn lại trong stack vào kết quả.

Ví dụ với biểu thức:

$$3 + 4 \times 2 - (-5 + 3)$$

Sau khi áp dụng thuật toán, ta được biểu thức Postfix như trong hình 2.10:

$$3\ 4\ 2\ *\ +\ -\ 5\ 3\ +\ -$$



Hình 2.10: Bước 2: Chuyển đổi từ Infix sang Postfix

Bước 3: Đánh giá biểu thức Postfix

Để tính toán biểu thức ở dạng **Postfix**, ta sử dụng một ngăn xếp (stack). Thuật toán:

- Duyệt từ trái sang phải các phần tử trong biểu thức.
- Nếu gặp toán hạng $\rightarrow$ đẩy vào stack.
- Nếu gặp toán tử $\rightarrow$ lấy ra 2 phần tử trên cùng của stack, áp dụng phép toán, rồi đẩy kết quả trở lại stack.
- Sau khi duyệt hết $\rightarrow$ giá trị cuối cùng trong stack chính là kết quả.

Ví dụ với Postfix:

3 4 2 * + - 5 3 + -

Ta đánh giá theo từng bước:

Token	Stack	Giải thích
3	[3]	Push 3
4	[3, 4]	Push 4
2	[3, 4, 2]	Push 2
*	[3, 8]	Pop 4, 2 $\rightarrow 4 * 2 = 8$
+	[11]	Pop 3, 8 $\rightarrow 3 + 8 = 11$
-5	[11, -5]	Push -5
3	[11, -5, 3]	Push 3
+	[11, -2]	Pop -5, 3 $\rightarrow -5 + 3 = -2$
-	[13]	Pop 11, -2 $\rightarrow 11 - (-2) = 13$

Kết quả cuối cùng:

$$3 + 4 \times 2 - (-5 + 3) = 13$$

Step 3: Postfix Evaluation

> Step-by-step Evaluation of Postfix

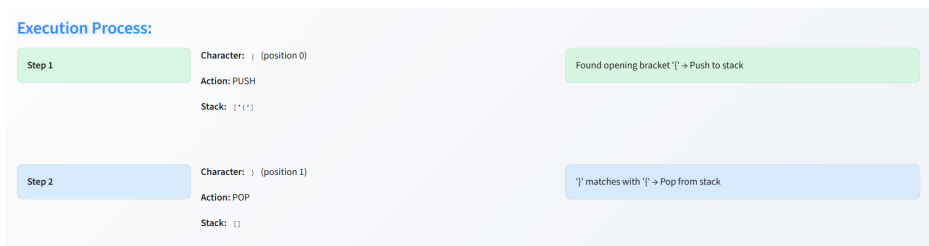
Hình 2.11: Bước 3: Đánh giá biểu thức Postfix

2.4 Kiểm tra dấu ngoặc

Nguyên lý thuật toán

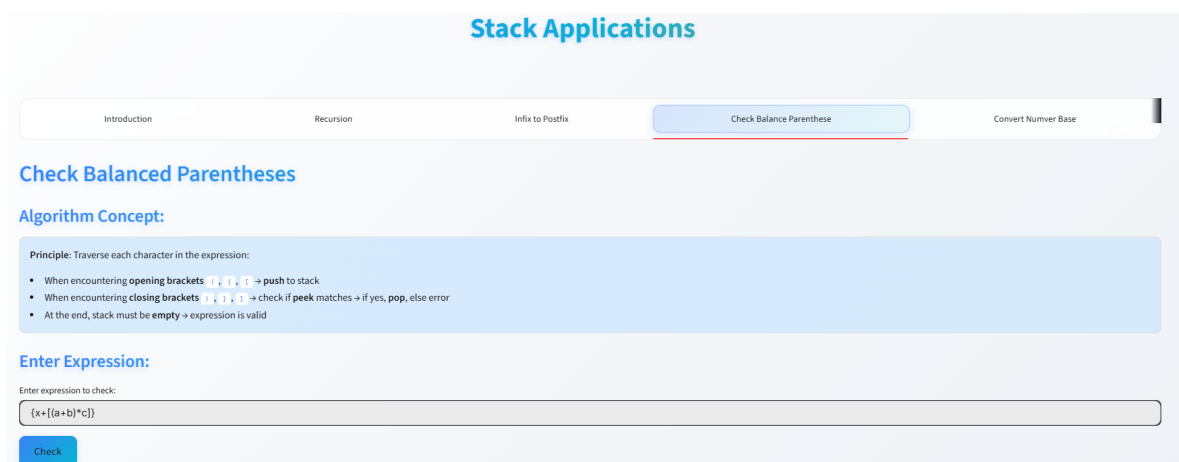
Hàm xử lý sẽ duyệt từng ký tự trong biểu thức và sử dụng stack để kiểm tra các dấu ngoặc theo phương pháp:

- Khi gặp dấu ngoặc mở (, {, [$\rightarrow$ push vào stack.
- Khi gặp dấu ngoặc đóng), },] $\rightarrow$ kiểm tra phần tử trên cùng (peek) của stack:
 - Nếu khớp loại ngoặc $\rightarrow$ pop ra khỏi stack.
 - Nếu không khớp hoặc stack rỗng $\rightarrow$ biểu thức sai.



Hình 2.13: Ví dụ minh họa kiểm tra dấu ngoặc nhọn

- Sau khi duyệt hết, nếu stack rỗng → biểu thức hợp lệ (ngoặc cân bằng).



Hình 2.12: Ứng dụng Stack: Kiểm tra ngoặc cân bằng

Ví dụ kiểm tra

Step 1: Gặp dấu ngoặc mở { → đưa vào stack.

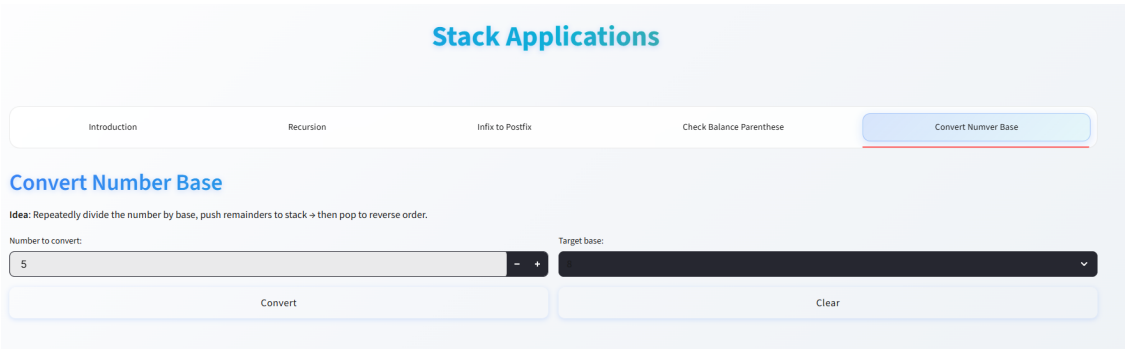
Step 2: Gặp dấu ngoặc đóng } → khớp với ngoặc mở trước đó, lấy ra khỏi stack.

Kết quả: Stack rỗng → biểu thức hợp lệ.

2.5 Đổi cơ số

Bằng cách lặp lại phép chia số cần đổi cho cơ số mới, đưa phần dư vào stack, rồi lấy ra (pop) để đảo ngược thứ tự và thu được kết quả.

- Nhập số cần đổi (ví dụ: 5).
- Chọn cơ số đích (ví dụ: 2).
- Thực hiện chia liên tiếp và lưu phần dư vào stack.
- Lấy các phần tử trong stack ra để được số ở hệ mới.



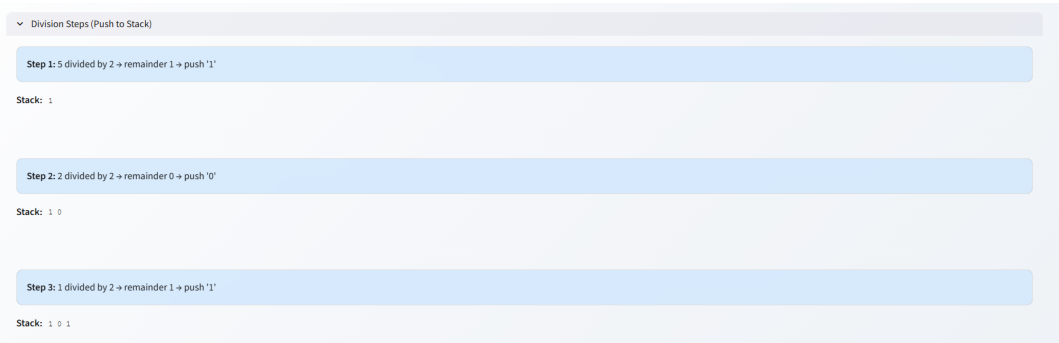
Hình 2.14: Giao diện ứng dụng Stack để đổi cơ số

Quy trình đổi số 5 (thập phân) sang nhị phân bằng stack:

Division Steps (Push to Stack):

- 1. $5 \div 2 = 2$, dư 1 $\rightarrow$ push 1.
- 2. $2 \div 2 = 1$, dư 0 $\rightarrow$ push 0.
- 3. $1 \div 2 = 0$, dư 1 $\rightarrow$ push 1.

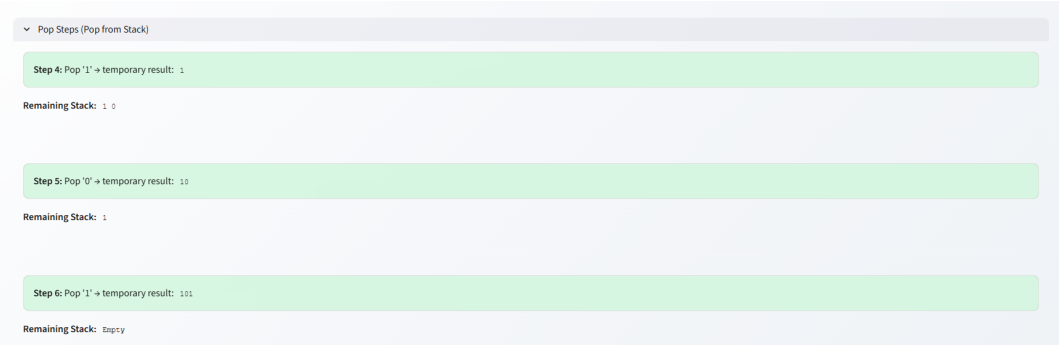
Stack cuối: 1, 0, 1.



Pop Steps (Pop from Stack):

- 1. Pop 1 $\rightarrow$ kết quả 1.
- 2. Pop 0 $\rightarrow$ kết quả 10.
- 3. Pop 1 $\rightarrow$ kết quả 101.

Stack rỗng, kết quả cuối: **101**.



KẾT LUẬN

3.1 Các kết quả đạt được

Qua quá trình xây dựng trang web và minh họa cơ bản các ứng dụng của stack, ứng dụng đã cho thấy rõ ràng nguyên lý hoạt động của cấu trúc dữ liệu **Stack** (ngăn xếp) với các thao tác cơ bản: Push, Pop, Peek, kiểm tra rỗng, xác định kích thước, và xóa toàn bộ phần tử. Các ví dụ minh họa như: **đệ quy (recursion)**, **chuyển đổi biểu thức Infix $\rightarrow$ Postfix**, **kiểm tra dấu ngoặc**, và **đổi cơ số** đã thể hiện tính trực quan và hiệu quả của Stack trong thực tế. Nhờ đó, người sử dụng có thể dễ dàng hình dung cách mà Stack được sử dụng trong giải quyết bài toán lập trình, cũng như nắm rõ nguyên tắc cơ bản của nó.

3.2 Hướng phát triển trong tương lai

Trong tương lai, ứng dụng có thể được phát triển theo các hướng sau:

- Mở rộng thêm nhiều ví dụ thực tế khác sử dụng Stack (ví dụ: Undo/Redo trong soạn thảo văn bản, duyệt cây nhị phân).
- Tích hợp minh họa trực quan hơn, kèm theo hoạt ảnh mô phỏng từng bước push/pop.
- Phát triển thêm các cấu trúc dữ liệu khác để tạo thành bộ công cụ trực quan toàn diện.
- Tối ưu giao diện người dùng để dễ dàng sử dụng trong giảng dạy và tự học.

PHỤ LỤC

Viết tắt	Giải thích
LIFO	Last In – First Out (Nguyên tắc vào sau ra trước)
GUI	Graphical User Interface (Giao diện đồ họa người dùng)
API	Application Programming Interface (Giao diện lập trình ứng dụng)

Danh sách hình vẽ

2.1	Hình minh họa trang Web	4
2.2	Thông báo log khi Push vào Stack	5
2.3	Thông tin về Stack	5
2.4	Xếp chồng các phần tử trong Call Stack	6
2.5	Giao diện đệ quy	7
2.6	Giai đoạn 1: Xây dựng call stack cho bài toán tính $5!$	7
2.7	Đệ quy đạt được điều kiện cơ sở: $factorial(1) = 1$	8
2.8	Chuyển đổi từ Infix sang Postfix bằng Stack	9
2.9	Bước 1: Phân tích biểu thức thành các token	9
2.10	Bước 2: Chuyển đổi từ Infix sang Postfix	10
2.11	Bước 3: Đánh giá biểu thức Postfix	11
2.13	Ví dụ minh họa kiểm tra dấu ngoặc nhọn	12
2.12	Ứng dụng Stack: Kiểm tra ngoặc cân bằng	12
2.14	Giao diện ứng dụng Stack để đổi cơ số	13

Ứng dụng được viết trên ngôn ngữ python và được chia thành nhiều file để dễ quản lý:

- `main.py`: Tập chính, quản lý luồng xử lý và giao diện tổng thể.
- `introduction.py`: Chứa nội dung giới thiệu lý thuyết về Stack.
- `recursion.py`: Minh họa hoạt động của đệ quy bằng Stack.
- `infix_2_postfix.py`: Chuyển đổi biểu thức từ Infix sang Postfix bằng Stack.
- `parenthese.py`: Kiểm tra độ chính xác ngoặc.
- `number_base.py`: Đổi cơ số.
- `theme.py`: Thiết lập giao diện, màu sắc và định dạng hiển thị.
- `__init__.py`: Đánh dấu thư mục như một gói (package) Python.

`main.py`

```
1 import streamlit as st
2 import theme, introduction, infix_2_postfix, number_base, parenthese,
  recursion
3 from theme import THEME_DARKMODE
4
5 st.set_page_config(
6     page_title="Stack Application",
7     layout="wide",
8     initial_sidebar_state="expanded",
9 )
10
11 st.markdown(theme.get_css(THEME_DARKMODE), unsafe_allow_html=True)
12
13 st.markdown("""
14     <div style="text-align: center;">
15         <h1>Stack Applications</h1>
16     </div>
17     """, unsafe_allow_html=True)
18 st.markdown("<br><br>", unsafe_allow_html=True)
```

```

19
20 # Create tabs
21 tab1, tab2, tab3, tab4, tab5 = st.tabs(['Introduction',
22                                         'Recursion',
23                                         'Infix to Postfix',
24                                         'Check Balance Parenthese',
25                                         'Convert Numver Base'])
26
27 with tab1:
28     introduction.render_stack_tab()
29
30 with tab2:
31     recursion.render_stack_tab()
32
33 with tab3:
34     infix_2_postfix.render_stack_tab()
35
36 with tab4:
37     parenthese.render_stack_tab()
38
39 with tab5:
40     number_base.render_stack_tab()

```

Listing 1: File main.py

infix_2_postfix.py

```

1 import streamlit as st
2 import time
3 from typing import List
4
5 def is_operator(token: str) -> bool:
6     return len(token) == 1 and token in "+-*/"
7
8 def is_valid_number(token: str) -> bool:
9     if not token:
10         return False
11     if token[0] == '-' and len(token) > 1:
12         return token[1:].isdigit()
13     return token.isdigit()

```

```

14
15 def tokenize(expression: str) -> List[str]:
16     tokens = []
17     index = 0
18     length = len(expression)
19
20     while index < length:
21         # Skip whitespace
22         if expression[index].isspace():
23             index += 1
24             continue
25
26         # Handle numbers (including negative numbers)
27         if expression[index].isdigit() or (
28             expression[index] == '-'
29             and index + 1 < length
30             and expression[index + 1].isdigit()
31         ):
32             number = ""
33             if expression[index] == '-':
34                 number += '-'
35                 index += 1
36             while index < length and expression[index].isdigit():
37                 number += expression[index]
38                 index += 1
39             tokens.append(number)
40
41         else:
42             # Handle unary minus: e.g., "-x" or "(-5)"
43             if expression[index] == '-':
44                 if not tokens or tokens[-1] == "(" or is_operator(tokens
45                     [-1]):
46                     tokens.append("0")
47                     tokens.append("-")
48                     index += 1
49                     continue
50
51             # Default: treat as a single-character token (operator,
52                 parenthesis, etc.)
53             tokens.append(expression[index])

```

```

52         index += 1
53
54     return tokens
55
56 def precedence(op: str) -> int:
57     if op in "+-":
58         return 1
59     if op in "*/":
60         return 2
61     return 0
62
63 def infix_to_postfix(infix: str) -> str:
64     tokens = tokenize(infix)
65     stack = []
66     output = []
67     with st.expander(f"Infix to postfix step-by-step", expanded=False):
68         for step, token in enumerate(tokens, 1):
69             if is_valid_number(token) or token.isidentifier():
70                 output.append(token)
71             elif token == "(":
72                 stack.append(token)
73             elif token == ")":
74                 while stack and stack[-1] != "(":
75                     output.append(stack.pop())
76                 if not stack:
77                     raise ValueError("Mismatched parentheses")
78                 stack.pop() # remove '('
79             elif is_operator(token):
80                 while (
81                     stack
82                     and is_operator(stack[-1])
83                     and precedence(stack[-1]) >= precedence(token)
84                 ):
85                     output.append(stack.pop())
86                 stack.append(token)
87             else:
88                 raise ValueError(f"Invalid token: {token}")
89
90     st.write(f"**Step** {step}: Read token '{token}'")
91

```

```
92         col1, col2 = st.columns(2)
93         with col1:
94             st.markdown("**Output:**")
95             st.code(" ".join(output) if output else "", language=None)
96         with col2:
97             st.markdown("**Stack:**")
98             st.code(" ".join(stack) if stack else "", language=None)
99         st.divider()
100         time.sleep(1.5)
101
102     while stack:
103         if stack[-1] in "()":
104             raise ValueError("Mismatched parentheses")
105         output.append(stack.pop())
106
107     result = " ".join(output)
108
109     return result
110
111 def evaluate_postfix(postfix: str) -> int:
112     stack = []
113     tokens = postfix.split()
114
115     with st.expander("Step-by-step Evaluation of Postfix", expanded=False):
116         :
117         for i, token in enumerate(tokens, 1):
118             action = ""
119
120             if is_valid_number(token):
121                 num = int(token)
122                 stack.append(num)
123                 action = f"Push number {num}"
124
125             elif is_operator(token):
126                 if len(stack) < 2:
127                     raise ValueError("Invalid postfix expression: not
128                                     enough operands")
129
130                 b = stack.pop()
131                 a = stack.pop()
132                 if token == "+":
```

```

130         res = a + b
131     elif token == "-":
132         res = a - b
133     elif token == "*":
134         res = a * b
135     elif token == "/":
136         if b == 0:
137             raise ValueError("Division by zero")
138         res = a // b
139     stack.append(res)
140     action = f"Pop {a}, {b}      {a} {token} {b} = {res}"
141         Push {res}"
142
143     else:
144         if token.isidentifier():
145             stack.append(0)
146             action = f"Push variable '{token}' (assumed value = 0)"
147
148         else:
149             raise ValueError(f"Invalid token in postfix: {token}")
150
151     st.markdown(f"**Step {i}**: Token = '{token}'")
152     st.caption(action)
153     st.write("**Stack:**", stack)
154
155     st.divider()
156     time.sleep(1.5)
157
158     if len(stack) != 1:
159         raise ValueError("Invalid postfix expression: too many/few"
160             operands")
161
162     result = stack[0]
163
164     return result
165
166 def render_stack_tab():
167     st.header("Infix to Postfix Expression Evaluator")
168     st.write(" ")

```

```

167     This program:
168     - Parses an infix expression (e.g., '3 + 4 * 2')
169     - Converts it to postfix notation (e.g., '3 4 2 * +')
170     - Evaluates the result step by step
171     - Handles negative numbers, unary minus (e.g., '-5', '(-x)'), and
      variables
172     """
173
174     expr = st.text_input("Enter an infix expression:", value="3 + 4 * 2 -
      (-5 + 3)")
175
176     if st.button("Convert and Calculate", key="calc_infix"):
177         if not expr.strip():
178             st.error("Please enter a valid expression!")
179             return
180
181         try:
182             st.subheader("Step 1: Tokenization")
183             tokens = tokenize(expr)
184             with st.expander("View Tokens", expanded=False):
185                 st.write("**Token list:**", tokens)
186                 st.code(" ".join(f"'{t}'" for t in tokens), language="text")
187
188             st.subheader("Step 2: Infix      Postfix Conversion")
189             postfix = infix_to_postfix(expr, )
190             st.code(postfix, language="text")
191
192             st.subheader("Step 3: Postfix Evaluation")
193             result = evaluate_postfix(postfix, )
194
195             st.divider()
196             st.subheader("Final Result")
197             st.success(f"**{expr} = {result}**")
198
199         except Exception as e:
200             st.error(f"**Error:** {str(e)}")
201             st.info("Check for mismatched parentheses, invalid tokens, or
      division by zero.")

```

Listing 2: File infix_2_postfix.py

number_base.py

```
1 import streamlit as st
2 import time
3
4 def convert_base_with_steps(num, base):
5     if num == 0:
6         return "0", [{
7             "action": "Number is 0      result is '0'",
8             "stack": [], "result": "0"
9         }]
10
11     digits = "0123456789ABCDEF"
12     stack = []
13     steps = []
14     n = num
15
16     # Division process
17     while n > 0:
18         remainder = n % base
19         digit = digits[remainder]
20         stack.append(digit)
21         step = {
22             'action': f"{n} divided by {base}      remainder {remainder}
23             push '{digit}'",
24             'stack': stack.copy(),
25             'result': ''
26         }
27         steps.append(step)
28         n //= base
29
30     # Pop process
31     result = ""
32     temp_stack = stack.copy()
33     for _ in range(len(stack)):
34         digit = temp_stack.pop()
35         result += digit
36         step = {
37             'action': f"Pop '{digit}'      temporary result: '{result}'",
38             'stack': temp_stack.copy(),
```

```

38         'result': result
39     }
40     steps.append(step)
41
42     return result, steps
43
44 def render_stack_tab():
45     st.header("Convert Number Base")
46     st.write("""
47     **Idea**: Repeatedly divide the number by base, push remainders to
48     stack then pop to reverse order.
49     """)
50     col_num, col_base = st.columns(2)
51     with col_num:
52         number = st.number_input("Number to convert:", min_value=0,
53                                 max_value=10000, value=233, step=1, key="num_input")
54     with col_base:
55         base = st.selectbox("Target base:", options=[2, 8, 10, 16], index
56                             =0, key="base_select")
57
58     col_btn3, col_btn4 = st.columns([1, 1])
59     with col_btn3:
60         convert_btn = st.button("Convert", key="convert_base",
61                                 use_container_width=True)
62     with col_btn4:
63         clear_btn2 = st.button("Clear", key="clear_base",
64                                 use_container_width=True)
65
66     if clear_btn2:
67         if 'base_steps' in st.session_state:
68             del st.session_state['base_steps']
69         if 'base_result' in st.session_state:
70             del st.session_state['base_result']
71         st.rerun()
72
73     if convert_btn:
74         result, steps = convert_base_with_steps(number, base)
75         # st.write(steps)
76         st.session_state['base_steps'] = steps
77         st.session_state['base_result'] = result

```

```

73
74     if 'base_steps' in st.session_state and 'base_result' in st.
       session_state:
75         steps = st.session_state['base_steps']
76         result = st.session_state['base_result']
77
78         # Display step-by-step
79         st.write("### Step-by-step Process:")
80
81         with st.expander("Division Steps (Push to Stack)", expanded=True):
82             for i, step in enumerate(steps):
83                 if "divided by" in step['action']:
84                     st.info(f"**Step {i+1}:** {step['action']}")
85                     st.write(f"**Stack:** {' '.join(step['stack'])}")
86                     st.divider()
87                     time.sleep(1.5)
88
89         with st.expander("Pop Steps (Pop from Stack)", expanded=True):
90             for i, step in enumerate(steps):
91                 if "Pop" in step['action']:
92                     st.success(f"**Step {i+1}:** {step['action']}")
93                     st.write(f"**Remaining Stack:** {' '.join(step['stack']
94                                     ')] if step['stack'] else 'Empty'}")
95                     st.divider()
96                     time.sleep(1.5)
97
98         st.success(f"### Final Result: '{number}' (base 10) = {result} (
99                     base {base})")

```

Listing 3: File number_base.py

parenthese.py

```

1 import streamlit as st
2 import time
3
4 def check_balanced_brackets(expr):
5     if not expr.strip():
6         return True, [{"message": "Empty expression - valid"}]
7

```

```

8 stack = []
9 bracket_pairs = {'(': ')', '[': ']', '{': '}'}
10 steps = []
11
12 for i, char in enumerate(expr):
13     # Skip non-bracket characters
14     if char not in "(){}[]":
15         continue
16
17     step_info = {
18         'step_num': len(steps) + 1,
19         'char': char,
20         'position': i,
21         'stack_before': stack.copy(),
22     }
23
24     # Opening bracket - push to stack
25     if char in "([[" :
26         stack.append(char)
27         step_info['action'] = 'PUSH'
28         step_info['status'] = 'push'
29         step_info['message'] = f"Found opening bracket '{char}' ->
30             Push to stack"
31
32     # Closing bracket - check and pop
33     else:
34         if not stack:
35             step_info['action'] = 'ERROR'
36             step_info['status'] = 'error'
37             step_info['message'] = f"Error: Found closing bracket '{
38                 char}' but stack is empty!"
39             step_info['stack_after'] = stack.copy()
40             steps.append(step_info)
41             return False, steps
42
43         if stack[-1] != bracket_pairs[char]:
44             step_info['action'] = 'ERROR'
45             step_info['status'] = 'error'
46             step_info['message'] = f"Error: '{char}' does not match
47                 with '{stack[-1]}'"

```

```

45         step_info['stack_after'] = stack.copy()
46         steps.append(step_info)
47         return False, steps
48
49         matched = stack.pop()
50         step_info['action'] = 'POP'
51         step_info['status'] = 'pop'
52         step_info['message'] = f"'{char}' matches with '{matched}' ->
        Pop from stack"
53
54         step_info['stack_after'] = stack.copy()
55         steps.append(step_info)
56
57     # Final check
58     is_valid = len(stack) == 0
59     steps.append({
60         'step_num': len(steps) + 1,
61         'char': 'END',
62         'action': 'CHECK',
63         'status': 'success' if is_valid else 'error',
64         'message': 'Stack is empty -> Expression is valid!' if is_valid
        else 'Stack is not empty -> Expression is invalid!',
65         'stack_after': stack.copy()
66     })
67
68     return is_valid, steps
69
70
71 def render_stack_tab():
72     st.header("Check Balanced Parentheses")
73
74     # Algorithm explanation
75     st.write("### Algorithm Concept:")
76     st.info("""
77     **Principle**: Traverse each character in the expression:
78     - When encountering **opening brackets** '(', '{', '[' -> **push** to
        stack
79     - When encountering **closing brackets** ')', '}', ']' -> check if **
        peek** matches -> if yes, **pop**, else error
80     - At the end, stack must be **empty** -> expression is valid

```

```
81     """)
82
83     # Input section
84     st.write("### Enter Expression:")
85     expression = st.text_input(
86         "Enter expression to check:",
87         value="{x+[(a+b)*c]}",
88         key="expr_input",
89         help="Examples: (a+b)*[c-d], {x+(y-z)}, [(a+b)]"
90     )
91
92     # Control button
93     check_btn = st.button("Check", key="check_brackets", type="primary")
94
95     # Main processing
96     if check_btn:
97         if not expression.strip():
98             st.warning("Please enter an expression to check!")
99             return
100
101         # Perform check and save to session state
102         is_valid, steps = check_balanced_brackets(expression)
103         st.session_state['bracket_steps'] = steps
104         st.session_state['bracket_result'] = is_valid
105         st.session_state['bracket_expr'] = expression
106
107     # Display results if available
108     if 'bracket_steps' in st.session_state:
109         st.write("### Execution Process:")
110
111         for step in st.session_state['bracket_steps']:
112             if step.get('char') == 'END':
113                 # Final step
114                 if step['status'] == 'success':
115                     st.success(f"**Step {step['step_num']}**": {step['message']})
116                 else:
117                     st.error(f"**Step {step['step_num']}**": {step['message']})
```

```

118         remaining = step['stack_after'] if step['stack_after']
119             else []
120         st.write(f"**Remaining Stack:** '{remaining}'")
121     else:
122         # Character processing steps
123         col1, col2, col3 = st.columns([1, 2, 2])
124
125         with col1:
126             if step['status'] == 'push':
127                 st.success(f"**Step {step['step_num']}**")
128             elif step['status'] == 'pop':
129                 st.info(f"**Step {step['step_num']}**")
130             else:
131                 st.error(f"**Step {step['step_num']}**")
132
133         with col2:
134             st.write(f"**Character:** '{step['char']}' (position {
135                 step['position']})")
136             st.write(f"**Action:** {step['action']}")
137             st.write(f"**Stack:** '{step['stack_after']}'")
138
139         with col3:
140             if step['status'] == 'push':
141                 st.success(step['message'])
142             elif step['status'] == 'pop':
143                 st.info(step['message'])
144             else:
145                 st.error(step['message'])
146                 break
147
148         st.divider()
149         time.sleep(1.5)
150     # Final result
151     st.write("### Result:")
152     expr = st.session_state.get('bracket_expr', expression)
153     if st.session_state.get('bracket_result'):
154         st.success(f"The expression '{expr}' is valid")
155     else:
156         st.error(f"The expression '{expr}' is invalid")

```

Listing 4: File parentheses.py

recursion.py

```
1 import streamlit as st
2 import time
3
4 def render_stack_tab():
5     st.header("Recursion")
6     st.write("Recursion is a programming technique where a function calls
7         itself to solve a problem.")
8     st.write("The stack is used to store function calls during recursion."
9         )
10
11     factorial_n = st.number_input("Enter n to calculate factorial:",
12         min_value=0, max_value=10, value=5)
13
14     col1, col2 = st.columns([5, 5])
15
16     # Initialize recursion stack in session state
17     if "recursion_stack" not in st.session_state:
18         st.session_state['recursion_stack'] = []
19
20     if "is_calculating" not in st.session_state:
21         st.session_state['is_calculating'] = False
22
23     if "final_result" not in st.session_state:
24         st.session_state['final_result'] = None
25
26     if "current_phase" not in st.session_state:
27         st.session_state['current_phase'] = 0
28
29     if "stack_built" not in st.session_state:
30         st.session_state['stack_built'] = False
31
32     with col1:
33         if st.button("Calculate Factorial", key="calc_factorial"):
34             st.session_state['is_calculating'] = True
35             st.session_state['recursion_stack'] = []
36             st.session_state['final_result'] = None
37             st.session_state['current_phase'] = 1
38             st.session_state['stack_built'] = False
```



```

36         st.rerun()
37
38     # Placeholder for stack visualization
39     stack_placeholder = col2.empty()
40
41     if st.session_state['is_calculating']:
42         placeholder = col1.empty()
43
44     # Phase 1: Building Call Stack
45     if st.session_state['current_phase'] == 1 and not st.session_state
46         ['stack_built']:
47         with col1:
48             st.write("**Phase 1: Building Call Stack**")
49
50     # Build stack from factorial(n) down to factorial(1)
51     for i in range(factorial_n, 0, -1):
52         call_info = {
53             'function': f'factorial({i})',
54             'parameter': i,
55             'status': 'waiting',
56             'result': None,
57             'depth': factorial_n - i
58         }
59         st.session_state['recursion_stack'].append(call_info)
60
61     # Update stack visualization
62     with stack_placeholder.container():
63         st.write("**Call Stack**")
64         for idx, call in enumerate(reversed(st.session_state['
65             recursion_stack'])):
66             expander_title = f"{call['function']} (TOP)" if
67                 idx == 0 else call['function']
68             expanded_fl = (idx == 0)
69
70             with st.expander(expander_title, expanded=
71                 expanded_fl):
72                 st.write(f"**Function:** '{call['function']}'"
73                     )
74                 st.write(f"**Parameter:** {call['parameter']}"
75                     )

```

```

70         st.write(f"**Status:** {call['status'].title()}")
71         st.write(f"**Call Depth:** {call['depth']}")
72         st.write(f"**Result:** Pending...")
73
74         # Status update
75         placeholder.info(f"Pushing factorial({i}) to stack...")
76         time.sleep(5)
77
78     placeholder.empty()
79     with col1:
80         st.success("All function calls pushed to stack!")
81
82     st.session_state['stack_built'] = True
83     st.session_state['current_phase'] = 2
84     time.sleep(1)
85     st.rerun()
86
87     # Phase 2: Calculating and Returning
88     elif st.session_state['current_phase'] == 2:
89         with col1:
90             st.write("**Phase 2: Calculating and Returning**")
91
92             # Initialize result_map if not exists
93             if 'result_map' not in st.session_state:
94                 st.session_state['result_map'] = {}
95
96             # Initialize current step in phase 2
97             if 'phase2_step' not in st.session_state:
98                 st.session_state['phase2_step'] = 1
99
100             current_n = st.session_state['phase2_step']
101
102             # If not exceeding n
103             if current_n <= factorial_n:
104                 idx_in_stack = factorial_n - current_n
105                 call = st.session_state['recursion_stack'][idx_in_stack]
106
107                 # Calculate result
108                 if current_n == 1:

```

```

109         result = 1
110         calc_text = "Base case: factorial(1) = 1"
111     else:
112         prev_result = st.session_state['result_map'][current_n
113             - 1]
114         result = current_n * prev_result
115         calc_text = f"factorial({current_n}) = {current_n}
116             {prev_result} = {result}"
117
118     st.session_state['result_map'][current_n] = result
119     st.session_state['recursion_stack'][idx_in_stack]['status']
120     ] = 'calculating'
121     st.session_state['recursion_stack'][idx_in_stack]['result']
122     ] = result
123
124     # Update stack visualization
125     stack_placeholder.empty()
126     with stack_placeholder.container():
127         st.write("**Call Stack:**")
128         active_calls = [call for call in st.session_state['
129             recursion_stack'] if call['status'] != 'completed']
130
131         if not active_calls:
132             st.info("Stack is empty - All calls completed!")
133         else:
134             for _, call_inner in enumerate(reversed(
135                 active_calls)):
136                 if call_inner['parameter'] == current_n:
137                     expander_title = f"{call_inner['function']
138                         ']} - CALCULATING"
139                     expanded_fl = True
140                 else:
141                     expander_title = f"{call_inner['function']
142                         ']}"
143                     expanded_fl = False
144
145                 with st.expander(expander_title, expanded=
146                     expanded_fl):
147                     st.write(f"**Function:** '{call_inner['
148                         function']}'('")

```

```

139         st.write(f"**Parameter:** {call_inner['parameter']}")
140         st.write(f"**Status:** {call_inner['status'].title()}")
141
142         if call_inner['result'] is not None:
143             st.success(f"**Result:** {call_inner['result']}")
144             if call_inner['parameter'] == 1:
145                 st.write("**Calculation:** Base case returns 1")
146             else:
147                 prev = st.session_state['result_map'][call_inner['parameter'] - 1]
148                 st.write(f"**Calculation:** {call_inner['parameter']} factorial({call_inner['parameter']-1}) = {call_inner['parameter']} {prev} = {call_inner['result']}")
149             else:
150                 st.write("**Result:** Waiting...")
151
152         # Display status
153         placeholder.info(f"{calc_text}")
154         time.sleep(3)
155
156         # Mark as completed
157         st.session_state['recursion_stack'][idx_in_stack]['status'] = 'completed'
158         placeholder.success(f"Returned: factorial({current_n}) = {result}")
159
160         # Increment step and rerun for next iteration
161         st.session_state['phase2_step'] += 1
162         st.rerun()
163
164     else:
165         # All steps completed

```

```

166         st.session_state['final_result'] = st.session_state['
            result_map'][factorial_n]
167         st.session_state['is_calculating'] = False
168         st.session_state['current_phase'] = 0
169         del st.session_state['phase2_step']
170
171         # Display empty stack
172         stack_placeholder.empty()
173         with stack_placeholder.container():
174             st.write("**Call Stack:**")
175             st.info("Stack is empty - All function calls completed
                !")
176
177         with col1:
178             st.write(f"Final Result: {factorial_n}! = {st.
                session_state['final_result']}")
179
180     else:
181         with stack_placeholder:
182             st.info("Click **Calculate Factorial** to visualize recursion.
                ")
183
184     with col1:
185         if st.button("Clear", key="clear_recursion"):
186             st.session_state['recursion_stack'] = []
187             st.session_state['is_calculating'] = False
188             st.session_state['final_result'] = None
189             st.session_state['current_phase'] = 0
190             st.session_state['stack_built'] = False
191             if 'result_map' in st.session_state:
192                 del st.session_state['result_map']
193             if 'phase2_step' in st.session_state:
194                 del st.session_state['phase2_step']
195             st.rerun()

```

Listing 5: File recursion.py

introduction.py

```

1 import streamlit as st

```

```

2 import sys
3
4 def render_stack_tab():
5     st.header("Example of Stack")
6     instruction_ = "A stack is a data structure that operates on the " \
7     "LIFO (Last In First Out) principle. " \
8     "Like a stack of plates: the last plate placed will be the first one
9     taken " \
10    "and you can only operate at the top."
11    st.write(instruction_)
12
13    # Initialize stack in session state
14    if "array_stack" not in st.session_state:
15        st.session_state['array_stack'] = []
16
17    if "last_operation" not in st.session_state:
18        st.session_state['last_operation'] = ""
19
20    if "operation_result" not in st.session_state:
21        st.session_state['operation_result'] = ""
22
23    col1, col2 = st.columns([5, 5])
24
25    with col1:
26        st.write("**Operations:**")
27
28        # Input for new value
29        value = st.text_input("Enter value:", key="stack_input")
30
31        # Create buttons in columns for better layout
32        btn_col1, btn_col2, btn_col3, btn_col4, btn_col5, btn_col6 = st.
33        columns(6)
34
35        with btn_col1:
36            # Push operation
37            if st.button('Push', key='push', use_container_width=True):
38                if value and value.strip():
39                    st.session_state['array_stack'].append(value.strip())
40                    st.session_state['last_operation'] = f"[PUSH]: Added
41                    '{value.strip()}' to stack"

```

```
39         st.session_state['operation_result'] = "success"
40         st.rerun()
41     else:
42         st.session_state['last_operation'] = "[PUSH]: Please
43             enter a valid value"
44         st.session_state['operation_result'] = "error"
45
46 with btn_col2:
47     # Pop operation
48     if st.button('Pop', key='pop', use_container_width=True):
49         if st.session_state['array_stack']:
50             popped_value = st.session_state['array_stack'].pop()
51             st.session_state['last_operation'] = f"[POP]: Removed
52                 '{popped_value}' from stack"
53             st.session_state['operation_result'] = "success"
54             st.rerun()
55         else:
56             st.session_state['last_operation'] = "[POP]: Stack is
57                 empty, cannot pop"
58             st.session_state['operation_result'] = "error"
59
60 with btn_col3:
61     # Peek operation
62     if st.button('Peek', key='peek', use_container_width=True):
63         if st.session_state['array_stack']:
64             top_value = st.session_state['array_stack'][-1]
65             st.session_state['last_operation'] = f"[PEEK]: Top
66                 element is '{top_value}'"
67             st.session_state['operation_result'] = "info"
68         else:
69             st.session_state['last_operation'] = "[PEEK]: Stack is
70                 empty"
71             st.session_state['operation_result'] = "error"
72
73 with btn_col4:
74     # isEmpty operation
75     if st.button('isEmpty', key='is_empty', use_container_width=
76         True):
77         is_empty = (len(st.session_state['array_stack']) == 0)
78         if is_empty:
```

```

73         st.session_state['last_operation'] = "[IS_EMPTY]:
           Stack is empty"
74     else:
75         st.session_state['last_operation'] = "[IS_EMPTY]:
           Stack has elements"
76         st.session_state['operation_result'] = "info"
77
78     with btn_col5:
79         # Size operation
80         if st.button('Size', key='size', use_container_width=True):
81             size = len(st.session_state['array_stack'])
82             st.session_state['last_operation'] = f"[SIZE]: Stack size
               is {size}"
83             st.session_state['operation_result'] = "info"
84
85     with btn_col6:
86         # Clear stack operation
87         if st.button('Clear Stack', key='clear', use_container_width=
           True):
88             st.session_state['array_stack'] = []
89             st.session_state['last_operation'] = "[CLEAR]: Cleared
               entire stack"
90             st.session_state['operation_result'] = "success"
91
92     # Display last operation result
93     if st.session_state['last_operation']:
94         if st.session_state['operation_result'] == "success":
95             st.success(st.session_state['last_operation'])
96         elif st.session_state['operation_result'] == "error":
97             st.error(st.session_state['last_operation'])
98         else:
99             st.info(st.session_state['last_operation'])
100
101     with col2:
102         if st.session_state['array_stack']:
103             empty, col, empty = st.columns([1, 3, 1])
104             # Display stack elements using expanders from top to bottom
105             with col:
106                 st.markdown("<br>", unsafe_allow_html=True)

```



```

107         for i in range(len(st.session_state['array_stack']) - 1,
108                        -1, -1):
109
110             # Create expander title with TOP indicator
111             if i == len(st.session_state['array_stack']) - 1:
112                 expander_title = f"Element {i + 1} (TOP)"
113                 expander_expanded = True
114             else:
115                 expander_title = f"Element {i + 1}"
116                 expander_expanded = False
117
118             with st.expander(expander_title, expanded=
119                             expander_expanded):
120                 st.write(f"**Value:** '{element}'")
121                 st.write(f"**Index:** {i}")
122                 stack_len = len(st.session_state['array_stack'])
123                 if i == stack_len - 1:
124                     position_text = "Top of stack"
125                 else:
126                     position_text = f"{stack_len - 1 - i} from top
127                                     "
128
129                 st.write(f"**Position:** {position_text}")
130                 st.write(f"**Size:** {sys.getsizeof(element)}
131                         bytes")
132                 st.write(f"**Address:** 0x{id(element):x}")
133
134     else:
135         st.markdown("<br>", unsafe_allow_html=True)
136         st.info("Stack is empty - No elements")

```

Listing 6: File introduction.py

theme.py

```

1 from textwrap import dedent
2
3 THEME_DARKMODE = {
4     "bg1":     "#0f0f23",
5     "bg2":     "#1a1a2e",

```

```

6     "bg3":      "#191b1f",
7     "text":     "#E6F1FF",
8     "muted":    "#8892b0",
9     "accent":   "#f6f6f6",
10    "accent2":   "#00d4ff",
11    "gold":      "#ffd700",
12    "panel_alpha": "0.05",
13    "blur_px":   "10px",
14 }
15
16
17 def get_css(theme: dict) -> str:
18     t = theme
19
20     return dedent(f"""
21     <head>
22         <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/
23             libs/font-awesome/6.4.0/css/all.min.css">
24         <link href="https://fonts.googleapis.com/css2?family=Inter:
25             wght@300;400;500;600;700&display=swap" rel="stylesheet">
26     </head>
27
28     <style>
29     :root {{
30         --bg1: {t["bg1"]};
31         --bg2: {t["bg2"]};
32         --bg3: {t["bg3"]};
33         --text: {t["text"]};
34         --muted: {t["muted"]};
35         --accent: {t["accent"]};
36         --accent2: {t["accent2"]};
37         --gold: {t["gold"]};
38         --panel-alpha: {t["panel_alpha"]};
39         --blur: {t["blur_px"]};
40     }}
41
42     .stApp {{
43         background: linear-gradient(135deg, var(--bg1) 0%, var(--bg2) 50%,
44             var(--bg3) 100%);
45         font-family: 'Inter', sans-serif;

```

```

43     color: var(--text);
44 }}
45
46 .stApp::before {{
47     content: '';
48     position: fixed;
49     top: 0; left: 0; width: 100%; height: 100%;
50     background-image:
51         radial-gradient(2px 2px at 20px 30px, var(--accent),
52             transparent),
53         radial-gradient(2px 2px at 40px 70px, var(--accent2),
54             transparent),
55         radial-gradient(1px 1px at 90px 40px, var(--gold), transparent
56             ),
57         radial-gradient(1px 1px at 130px 80px, #ff6b6b, transparent);
58     animation: sparkle 20s linear infinite;
59     pointer-events: none;
60     z-index: 0;
61 }}
62
63 @keyframes sparkle {{
64     0% {{ transform: translateY(0px) rotate(0deg); opacity: 0.7; }}
65     50% {{ transform: translateY(-100px) rotate(180deg); opacity: 1;
66         }}
67     100% {{ transform: translateY(-200px) rotate(360deg); opacity:
68         0.7; }}
69 }}
70
71 .main-content {{
72     position: relative; z-index: 1;
73     backdrop-filter: blur(var(--blur));
74     background: rgba(255,255,255, var(--panel-alpha));
75     border-radius: 20px;
76     border: 1px solid rgba(255,255,255, 0.1);
77     padding: 2rem; margin: 1rem 0;
78     box-shadow: 0 8px 32px rgba(0, 0, 0, 0.3);
79 }}
80
81 .profile-section {{
82     background: linear-gradient(145deg, rgba(255,255,255,0.1), rgba
83         (255,255,255,0.05));

```

```
77     border-radius: 20px; padding: 2rem;
78     border: 1px solid rgba(255,255,255, 0.1);
79     box-shadow: 0 8px 32px rgba(0, 0, 0, 0.3);
80     backdrop-filter: blur(var(--blur));
81     position: relative; overflow: hidden;
82 }}
83 .profile-section::before {{
84     content: '';
85     position: absolute; top: -50%; left: -50%;
86     width: 200%; height: 200%;
87     background: conic-gradient(transparent, color-mix(in oklab, var(--
88         accent) 30%, transparent), transparent);
89     animation: rotate 10s linear infinite; z-index: -1;
90     opacity: 0.7;
91 }}
92 @keyframes rotate {{ 0% {{transform: rotate(0deg);}} 100% {{transform:
93     rotate(360deg);}} }}
94
95 h1, h2, h3, h4, h5, h6 {{
96     background: linear-gradient(135deg, var(--accent), var(--accent2),
97         var(--gold));
98     -webkit-background-clip: text; background-clip: text; -webkit-text
99     -fill-color: transparent;
100     font-weight: 700;
101     text-shadow: 0 0 30px color-mix(in oklab, var(--accent) 35%,
102         transparent);
103 }}
104 .main-title {{ font-size: 3.5rem !important; margin-bottom: .5rem !
105     important; animation: glow 2s ease-in-out infinite alternate; }}
106 .subtitle {{ font-size: 1.8rem !important; color: var(--accent) !
107     important; margin-bottom: 1rem !important; font-weight: 500 !
108     important; }}
109 @keyframes glow {{
110     from {{ text-shadow: 0 0 20px color-mix(in oklab, var(--accent)
111         30%, transparent); }}
112     to {{ text-shadow: 0 0 30px color-mix(in oklab, var(--accent) 60%,
113         transparent), 0 0 40px color-mix(in oklab, var(--accent) 30%,
114         transparent); }}
115 }}
```

```
106 .description {{
107     color: var(--muted);
108     font-size: 1.1rem; line-height: 1.8; margin-bottom: 2rem;
109     padding: 1rem; border-left: 3px solid var(--accent);
110     background: color-mix(in oklab, var(--accent) 10%, transparent);
111     border-radius: 0 10px 10px 0;
112 }}
113
114 .contact-section {{
115     background: color-mix(in oklab, var(--accent) 15%, transparent);
116     border-radius: 15px; padding: 1.5rem; margin: 2rem 0;
117     border: 1px solid color-mix(in oklab, var(--accent) 40%,
118         transparent);
119     box-shadow: 0 4px 20px color-mix(in oklab, var(--accent) 12%,
120         transparent);
121 }}
122 .contact-item {{
123     display: flex; align-items: center; margin: .8rem 0;
124     color: var(--text); font-size: 1rem; transition: all .3s ease;
125 }}
126 .contact-item:hover {{ color: var(--accent); transform: translateX(10
127     px); }}
128 .contact-item i {{ margin-right: 10px; color: var(--accent); width: 20
129     px; }}
130
131 .social-container {{
132     background: rgba(255,255,255,0.05);
133     border-radius: 15px; padding: 1.5rem; text-align: center;
134     border: 1px solid rgba(255,255,255,0.1); margin-top: 2rem;
135 }}
136 .social-icons {{ display: flex; justify-content: center; gap: 2rem;
137     padding: 1rem 0; }}
138 .social-icon {{
139     display: inline-block; width: 60px; height: 60px; line-height: 60
140     px; text-align: center;
141     background: linear-gradient(145deg, color-mix(in oklab, var(--
142         accent) 10%, transparent), color-mix(in oklab, var(--accent2)
143         10%, transparent));
144     border-radius: 50%; border: 2px solid transparent; background-clip
145         : padding-box;
```

```
137     transition: all .4s ease; position: relative; overflow: hidden;
138 }}
139 .social-icon::before {{
140     content: ''; position: absolute; inset: 0; border-radius: 50%;
141     padding: 2px;
142     background: linear-gradient(45deg, var(--accent), var(--accent2),
143     var(--gold));
144     -webkit-mask: linear-gradient(#fff 0 0) content-box, linear-
145     gradient(#fff 0 0);
146     -webkit-mask-composite: exclude; mask: linear-gradient(#fff 0 0)
147     content-box, linear-gradient(#fff 0 0);
148     mask-composite: exclude; opacity: 0; transition: opacity .4s ease;
149 }}
150 .social-icon:hover::before {{ opacity: 1; }}
151 .social-icon:hover {{ transform: translateY(-5px) scale(1.1); box-
152     shadow: 0 10px 30px color-mix(in oklab, var(--accent) 30%,
153     transparent); }}
154 .social-icon i {{ color: var(--muted); font-size: 1.5rem; transition:
155     all .3s ease; position: relative; z-index: 1; }}
156 .social-icon:hover i {{ color: var(--accent); transform: scale(1.2);
157     }}
158
159 .stTabs [data-baseweb="tab-list"] {{
160     display: flex; flex-wrap: nowrap; justify-content: space-between;
161     gap: 10px; width: 100%;
162     padding: 10px; border-radius: 16px;
163     background: rgba(255,255,255, var(--panel-alpha));
164     border: 1px solid rgba(255,255,255, 0.1);
165     backdrop-filter: blur(var(--blur));
166     overflow-x: auto; scrollbar-width: none;
167 }}
168 .stTabs [data-baseweb="tab"] {{
169     flex: 1 1 auto; min-width: 110px; height: 44px; padding: 0 16px;
170     border-radius: 12px; color: var(--muted); background: transparent;
171     border: 1px solid transparent; transition: all .25s ease;
172     font-weight: 600; letter-spacing: .2px;
173 }}
174 .stTabs [aria-selected="true"] {{
175     background: linear-gradient(135deg, color-mix(in oklab, var(--
176     accent) 20%, transparent), color-mix(in oklab, var(--accent2)
```

```
167         10%, transparent)) !important;
168         color: var(--accent) !important;
169         border: 1px solid color-mix(in oklab, var(--accent) 35%,
170             transparent) !important;
171         box-shadow: 0 4px 15px color-mix(in oklab, var(--accent) 20%,
172             transparent);
173     }}
174
175     .stTextInput > div > div > input,
176     .stNumberInput > div > div > input,
177     .stTextArea > div > div > textarea,
178     input[type="text"], input[type="number"], textarea {{
179         background: rgba(255,255,255,0.08) !important;
180         border: 1px solid rgba(255,255,255,0.15) !important;
181         border-radius: 10px !important;
182         color: var(--text) !important;
183         padding: 0.75rem 1rem !important;
184         font-size: 1rem !important;
185         font-family: 'Inter', sans-serif !important;
186         transition: all 0.3s ease !important;
187         backdrop-filter: blur(var(--blur)) !important;
188         box-shadow: 0 2px 8px rgba(0,0,0,0.1) !important;
189     }}
190
191     .stTextInput > div > div > input:focus,
192     .stNumberInput > div > div > input:focus,
193     .stTextArea > div > div > textarea:focus,
194     input[type="text"]:focus, input[type="number"]:focus, textarea:focus
195     {{
196         border: 2px solid var(--accent) !important;
197         box-shadow: 0 0 0 3px color-mix(in oklab, var(--accent) 20%,
198             transparent) !important;
199         outline: none !important;
200         transform: translateY(-1px) !important;
201     }}
```

```
202
203 .stButton button,
204 button[data-testid="baseButton-primary"],
205 button[data-testid="baseButton-secondary"] {{
206     background: rgba(255,255,255,0.0) !important;
207     color: #ffffff !important;
208     border: none !important;
209     border-radius: 10px !important;
210     padding: 0.75rem 1.5rem !important;
211     font-weight: 600 !important;
212     font-size: 1rem !important;
213     font-family: 'Inter', sans-serif !important;
214     transition: all 0.3s ease !important;
215     box-shadow: 0 4px 15px color-mix(in oklab, var(--accent) 25%,
216         transparent) !important;
217     text-transform: none !important;
218     letter-spacing: 0.5px !important;
219     cursor: pointer !important;
220 }}
221 .stButton button:hover,
222 button[data-testid="baseButton-primary"]:hover,
223 button[data-testid="baseButton-secondary"]:hover {{
224     transform: translateY(-2px) scale(1.02) !important;
225     box-shadow: 0 6px 20px color-mix(in oklab, var(--accent) 35%,
226         transparent) !important;
227     background: linear-gradient(135deg, var(--accent), var(--accent2))
228         !important;
229 }}
230 .stButton button:active,
231 button[data-testid="baseButton-primary"]:active,
232 button[data-testid="baseButton-secondary"]:active {{
233     transform: translateY(0) scale(1) !important;
234     box-shadow: 0 2px 8px color-mix(in oklab, var(--accent) 25%,
235         transparent) !important;
236 }}
237
238 .stCodeBlock, [data-testid="stCodeBlock"], .stMarkdown pre, div[data-
239     testid="stMarkdownContainer"] pre {{
240     background: rgba(255,255,255,0.08) !important;
241     border: 1px solid rgba(255,255,255,0.15) !important;
```



```
237     border-radius: 10px !important;
238 }}
239 .stCodeBlock code, [data-testid="stCodeBlock"] code, .stMarkdown pre
240     code, div[data-testid="stMarkdownContainer"] pre code {{
241     background: transparent !important;
242     color: var(--text) !important;
243 }}
244 code {{
245     background: rgba(255,255,255,0.08) !important;
246     color: var(--accent) !important;
247     padding: 0.2rem 0.4rem !important;
248     border-radius: 4px !important;
249     font-family: 'Courier New', monospace !important;
250 }}
251 .streamlit-expanderHeader {{
252     background: rgba(255,255,255,0.08) !important;
253     border: 1px solid rgba(255,255,255,0.15) !important;
254     border-radius: 10px !important;
255     color: var(--text) !important;
256 }}
257 .streamlit-expanderContent {{
258     background: rgba(255,255,255,0.08) !important;
259     border: 1px solid rgba(255,255,255,0.15) !important;
260     border-top: none !important;
261     border-radius: 0 0 10px 10px !important;
262     color: var(--text) !important;
263 }}
264 details[open] summary {{ border-radius: 10px 10px 0 0 !important; }}
265
266 pre, pre[class*="language-"] {{
267     background: rgba(255,255,255,0.08) !important;
268     border: 1px solid rgba(255,255,255,0.15) !important;
269     color: var(--text) !important;
270     padding: 1rem !important;
271 }}
272 code, code[class*="language-"] {{
273     background: transparent !important;
274     color: var(--text) !important;
275 }}
```

```
276
277 .stDownloadButton button {{
278     background: linear-gradient(135deg, var(--gold), #ffa000) !
           important;
279     color: white !important;
280 }}
281 .stDownloadButton button:hover {{
282     background: linear-gradient(135deg, #ffa000, var(--gold)) !
           important;
283     transform: translateY(-2px) scale(1.02) !important;
284 }}
285
286 button[data-baseweb="button"] {{
287     background: rgba(255,255,255,0.08) !important;
288     color: var(--text) !important;
289     border: 1px solid rgba(255,255,255,0.15) !important;
290     border-radius: 8px !important;
291     padding: 0.5rem 1rem !important;
292     margin: 0.25rem !important;
293     transition: all 0.3s ease !important;
294     font-family: 'Inter', sans-serif !important;
295 }}
296 button[data-baseweb="button"]:hover {{
297     background: var(--accent) !important;
298     color: white !important;
299     transform: translateY(-1px) !important;
300     border-color: var(--accent) !important;
301 }}
302
303 .decoration-line {{
304     height: 2px;
305     background: linear-gradient(90deg, transparent, var(--accent), var
           (--accent2), transparent);
306     margin: 2rem 0; border-radius: 1px; animation: pulse-line 3s ease-
           in-out infinite;
307 }}
308 @keyframes pulse-line {{
309     0%, 100% {{ opacity: .5; transform: scaleX(1); }}
310     50% {{ opacity: 1; transform: scaleX(1.02); }}
311 }}
```

```
312
313 .floating-element {{
314     position: absolute; border-radius: 50%; opacity: 0.1; animation:
315         float 6s ease-in-out infinite;
316 }}
317 .floating-element:nth-child(1) {{ width: 80px; height: 80px; top: 10%;
318     right: 10%; background: var(--accent); animation-delay: 0s; }}
319 .floating-element:nth-child(2) {{ width: 60px; height: 60px; top: 60%;
320     right: 20%; background: var(--accent2); animation-delay: 2s; }}
321 .floating-element:nth-child(3) {{ width: 40px; height: 40px; top: 30%;
322     right: 5%; background: var(--gold); animation-delay: 4s; }}
323 @keyframes float {{
324     0%, 100% {{ transform: translateY(0px) rotate(0deg); }}
325     33% {{ transform: translateY(-20px) rotate(120deg); }}
326     66% {{ transform: translateY(10px) rotate(240deg); }}
327 }}
328 ::-webkit-scrollbar {{ width: 8px; }}
329 ::-webkit-scrollbar-track {{ background: rgba(255,255,255,0.1); border
330     -radius: 10px; }}
331 ::-webkit-scrollbar-thumb {{ background: linear-gradient(135deg, var
332     (--accent), var(--accent2)); border-radius: 10px; }}
333 ::-webkit-scrollbar-thumb:hover {{ background: linear-gradient(135deg,
334     var(--accent2), var(--accent)); }}
335
336 .stAlert {{
337     background: rgba(255,255,255,0.08) !important;
338     border: 1px solid rgba(255,255,255,0.15) !important;
339     border-radius: 10px !important;
340     color: var(--text) !important;
341     backdrop-filter: blur(var(--blur)) !important;
342 }}
343
344 p, span, div, label {{ color: var(--text) !important; }}
```

```
340 .stMetric {{
341     background: rgba(255,255,255,0.08) !important;
342     border: 1px solid rgba(255,255,255,0.15) !important;
343     border-radius: 10px !important;
344     padding: 1rem !important;
```

```
345         backdrop-filter: blur(var(--blur)) !important;
346     }}
347 </style>
348 """).strip()
```

Listing 7: File theme.py

